trigger:
- User asks for a Python report-generation script that extracts data from Oracle, transforms it with pandas, and writes a format
- User provides SQL, schema notes, mapping rules, or workbook layout requirements and wants implementation code.
non_trigger:
- User only wants SQL tuning, database performance diagnosis, or execution-plan analysis.
- User asks for non-Python report stacks (BI dashboards, VBA-only workflows, or JS/TypeScript pipelines).
failure_modes_addressed:
- Requirements gaps for workbook layout, naming, and formatting lead to unusable deliverables.
- Incomplete Oracle lifecycle handling causes leaked connections/cursors or brittle runtime behavior.
- Weak dtype/null handling in pandas causes incorrect aggregations, joins, or Excel outputs.
- Styling logic is mixed with data logic in ways that make the script hard to maintain.
attention_signals:
- Missing or ambiguous details about sheet names, headers, ordering, or formatting standards.
- Query output may be large enough to require chunking, filtering, or memory-aware transforms.
- Columns need explicit type conversion (dates, decimals, IDs with leading zeros, booleans).
- Consumer expects business-ready workbook polish (freeze panes, widths, number formats, table-like readability).
procedure:
- Confirm inputs and constraints first: SQL (or compressed schema + business logic), runtime environment, output path, and de
- Extract workbook requirements explicitly: sheet list, column order/header text, formatting preferences, sort/group rules, and v
- Plan script structure with clear stages: config/constants, Oracle extraction, pandas transforms, workbook writer, post-write sty
- Implement Oracle extraction with `python-oracledb` Thin mode, parameterized SQL, safe cursor usage, and deterministic clea
- Implement transformations with pandas using explicit dtype/null handling, joins/maps, derived columns, and stable ordering b
- Generate workbook output, then apply `openpyxl` styling in a separate pass so data and presentation concerns stay decoupled.
- Add robust error handling and logging surfaces, including connection failures, empty-result behavior, and file write errors.
- Verify locally with representative data and confirm workbook structure, formatting, and row/column integrity before handoff.
scope_boundary:
in_scope:
- Prompting Copilot to generate a maintainable Python script for Oracle-to-pandas-to-Excel reporting.
- Ensuring code includes extraction, transform, workbook writing, styling, cleanup, and local verification instructions.
out_of_scope:
- Editing SQL-tuning procedural assets or optimization playbooks outside this closet.
- Defining enterprise deployment infrastructure, schedulers, or CI/CD pipelines unless explicitly requested.
composition_points:
- Use this closet after requirements are available and before any SQL-tuner or deployment-specific procedural guidance.
- Pair with team standards for logging/config management when those standards are provided in the active task context.
reference_pointers:
- ref: python-idioms
section: pandas_etl
open_when: python script needs to query Oracle or load/map data to Pandas DataFrames.
- ref: python-idioms
section: openpyxl_excel
open_when: script needs to generate or apply formatting/styling to an Excel workbook.
verification:
- Confirm generated script runs locally with a sample or staging query and exits without leaked DB resources.
- Confirm workbook has expected sheets, headers, row counts, ordering, and formatting conventions.
- Confirm dtype conversions, null handling, and numeric/date formats match requirements.
output_contract:
- Deliver one coherent Python script (or clearly separated modules if requested) ready to run locally.
- Include concise run instructions, dependency assumptions, and required input placeholders.
- Preserve clear separation between extraction logic, transformation logic, and workbook styling logic.

Report Generator Closet

Use this closet to steer Copilot as a Senior Python Developer building pragmatic, maintainable report scripts.

Persona And Context Bootstrap

Start by framing Copilot as:

- Senior Python Developer focused on data reliability and Excel deliverable quality.
- Responsible for translating business reporting requirements into readable, testable script structure.
- Expected to prioritize explicit assumptions, safe database usage, and deterministic output.

Requirements Intake Checklist

Before writing code, collect and restate:

- Oracle extraction input:
  - Full SQL query, or compressed schema + join/filter/aggregation intent.
  - Bind parameters, date windows, and expected row-volume range.
- Output workbook definition:
  - Output file name/path, sheet names, and sheet ordering.
  - Required headers/column order and any renaming map.
  - Per-column formatting preferences (date, percent, currency, decimals, text preservation).
  - Styling expectations (header style, widths, freeze panes, filters, alignment, highlighting rules).
- Runtime constraints:
  - Python version, package constraints, credential source, and local execution environment.

If details are missing, ask targeted clarification questions before code generation.

Implementation Guidance

Direct Copilot to produce code with this control flow:

1. Configuration and constants (connection settings, SQL text, output paths, formatting maps).
2. Oracle extraction layer using `python-oracledb` Thin mode:
   - parameterized query execution,
   - predictable cursor/connection cleanup,
   - defensive handling for empty result sets and DB errors.
3. pandas transformation layer:
   - explicit dtype conversion,
   - null handling rules,
   - mappings/joins/derived fields,
   - memory-aware operations for larger datasets.
4. Excel output layer:
   - write DataFrames to sheets with stable ordering,
   - apply `openpyxl` formatting/styling in a dedicated pass.
5. Entrypoint + error handling:
   - clear exceptions/log messages,
   - non-zero exit on failure paths,
   - successful completion summary.

Keep data logic and styling logic separate, and avoid mixing SQL text assembly into transform code paths.

Local Verification Expectations

Require Copilot to include executable checks:

- Script runs end-to-end and closes DB resources cleanly.
- Produced workbook opens and matches requested sheet names, headers, order, and formatting.
- Row counts and key totals reconcile with extraction expectations.
- Date/number/text columns render correctly in Excel and preserve business meaning.

Output Expectations

Copilot should return:

- The Python script (or requested module split) with concise inline comments where intent is non-obvious.
- Minimal run instructions and dependency list.
- A short assumptions/limitations note for any unresolved requirement gaps.